

קורס NodeJS תשפו

Express

express היא חבילת npm שמאפשרת כתיבת server בnode בצורה קלה וגמישה. החבילה כוללת סט כלים רחב לניהול בקשות HTTP, כולל routing, קבלת פרמטרים, middlewares ועוד, תוך שמירה על קוד פשוט ונקי.

התקנה

```
npm i express
npm i -D @types/express // express type definitions
```

יצירת server בסיסי

```
import express, { Request, Response } from 'express';

const HOST = "127.0.0.1";
const PORT = 3000;

const app = express();

app.get('/', (req: Request, res: Response) => {
  res.send('Hello World!');
});

app.listen(PORT, HOST, () => {
  console.log(`App listening on http://${HOST}:${PORT}/`);
});
```

יצירת application של express נעשית באמצעות קריאה לפונקציה `express()`.

לאחר יצירת הapp ניתן להוסיף לו פונקציות לניהול בקשות שונות – לדוגמא, בקטע הקוד קיימת פונקציה לניהול בקשות המתקבלות בניתוב '/' עם method של GET, ומחזירה 'Hello World!'.

כדי להרים את app ולגרום לו להאזין לבקשות שיתקבלו בserver, יש לקרוא לפונקציה `listen` שמקבלת פרמטרים של `port`, `host` וcallback שירוך כשהapp יתחיל להאזין לבקשות.

Routing

הגדרת routing כוללת לכל URL HTTP Method פונקציונליות ייחודית.

בexpress, הגדרת route תיראה כך:

```
app.METHOD(PATH, HANDLER)
```

כאשר:

- app – application קיים של express.
- METHOD – HTTP method באותיות קטנות (לדוגמא: get, post, put, delete).
- PATH – הניתוב בserver.
- HANDLER – פונקציה שתרוץ כאשר תגיע לשרת בקשה שתואמת את הmethod והpath שהוגדרו.

דוגמאות:

ניהול של בקשות POST בניתוב '/':

```
app.post('/', (req: Request, res: Response) => {  
  res.send('Got a POST request');  
});
```

ניהול של בקשות PUT בניתוב 'user':

```
app.put('/user', (req: Request, res: Response) => {  
  res.send('Got a PUT request at /user');  
});
```

Request Parameters

Route Parameters

כדי להגדיר route שכולל פרמטרים (כמו ID של resource בserver), נוסף את שם הפרמטר כשלפניו ':' לניתוב. קבלת הפרמטרים בhandler היא באמצעות req.params

```
app.get('/users/:userId/books/:bookId', (req: Request, res: Response) => {
  const routeParams = req.params;
  const userId = routeParams.userId;
  const bookId = routeParams.bookId;

  console.log(`User: ${userId}, Book ${bookId}.`);
  res.send({userId, bookId});
});
```

לדוגמא - URL מתאים לroute:

GET <http://localhost:3000/users/34/books/8989>

התשובה שתחזור:

```
{ "userId": "34", "bookId": "8989" }
```

Query Parameters

פרמטרים שנשלחים על query (כמו פרמטרים לfilter או pagination) ניתן לקרוא בhandler באמצעות req.query

```
app.get('/books', (req: Request, res: Response) => {
  const category = req.query.category;
  res.send({books: [], category});
});
```

לדוגמא - URL מתאים לroute:

GET <http://localhost:3000/books?category=History>

התשובה שתחזור:

```
{ "books": [], "category": "History" }
```

Request Body

ניתן לקרוא את הdata שנשלח בbody של הבקשה, באמצעות req.body. כדי שהbody יגיע לאחר האיסוף של הdata בstream ויהיה ניתן לקרוא אותו כאובייקט יחיד, נוסיף את השורה הבאה: (נסביר אותה בהמשך)

```
app.use(express.json());
```

ובתוך handler:

```
app.post('/books', (req: Request, res: Response) => {  
  const book = req.body as Book;  
  // continue  
});
```

Response

אובייקט response מכיל פונקציות שונות שמעדכנות את response שיחזור מהAPI.

כל אחת מהפונקציות גורמות לניהול הבקשה להסתיים. ללא קריאה לאחת הפונקציות, הבקשה תישאר פתוחה ולא תיסגר – לא תוחזר תשובה לclient.

res.download()

הורדת קובץ:

```
res.download('/report-12345.pdf');
```

res.send()

שליחת הודעה כתשובה, תומך בtypes שונים של תשובות כמו string, json, array, boolean

```
res.send({ some: 'json' });
```

ניתן לשלב עם res.status() שמעדכן את status response:

```
res.status(404).send('Sorry, we cannot find that!');
```

```
res.status(500).send({ error: 'something blew up' });
```

res.redirect()

הפניה של הבקשה לכתובת אחרת:

```
res.redirect('http://example.com');
```

ה status code של redirect יהיה 302 בדרך כלל. כדי לשנות אותו ניתן להעביר status code אחר כפרמטר ראשון לפונקציה, לפני url.

`res.end()`

מסיים את ניהול הבקשה מבלי לשלוח הודעה ב body.

Router

כדי להפריד routes שונים לפי prefix של url ולקבץ routes ששייכים לאותו controller יחד, ניתן להשתמש ב router. router מתפקד כ mini-app – ניתן להגדיר עליו route handlers כפי שמגדירים על app עצמו.

בדוגמא הבאה, ניצור router לניהול routes שכלולים ב controller של birds:

```
import { Router } from 'express';

export const birdsRouter = Router();
birdsRouter.get('/', (req, res) => {
  res.send('Get all birds');
});

birdsRouter.get('/:id', (req, res) => {
  res.send('Get specific bird');
});

// all REST
```

לאחר יצירת ה router, נגדיר את router אליו ב app:

```
app.use('/birds', birdsRouter);
```

כעת, כל הבקשות שיגיעו לשרת עם כשה url prefix הוא /birds, ינווטו ל birdsRouter וינהלו בעזרת ה route handlers שהוגדרו עליו.

בעזרת routers, ניתן לנהל controllers שונים בשרת תוך הפרדה לוגית של resources וקיבוץ route handlers ששייכים לאותו resource יחד.

Middlewares

בהגדרת handlers לניהול בקשות שמגיעות לserver, נרצה לעיתים להגדיר middlewares – פונקציות אחידות לניהול גנרי של בקשות שונות לפני או אחרי הניהול הפרטני.

הגדרת middleware שימושית לניהול פרמטרים ובדיקות התקינות שלהם, logging של בקשות ופרטים שלהן, בדיקת הרשאות, הכנת תשובה ושליחה שלה ועוד.

בexpress, middleware היא פונקציה שמקבלת שלושה פרמטרים – request, response – שמייצג את הפונקציה הבאה בrequest chaining. middleware יכול לקרוא את הבקשה ולבצע בה שינויים, לעדכן את response, ולזמן את הפונקציה הבאה (שיכולה להיות middleware הבאה או route handler עצמו).

הוספת middleware היא באמצעות קריאה לפונקציה app.method – לניהול בקשות שמתקבלות בניית ספציפי http method מסוים, או app.use לניהול גנרי על כל הבקשות שמגיעות לשרת:

```
app.get('/', (req, res, next: NextFunction) => {
  console.log(`[${req.method}] ${req.url}`);
  next();
});
```

פונקציית middleware לעיתים קרובות תוגדר כפונקציה בקובץ utils, ותיקרא לניהול routes שונים בפרויקט:

```
export function parseUserId(req, res, next: NextFunction) {
  const userId = parseInt(req.params.userId);
  if (!userId) {
    res.status(400).end(`User id must be a number`);
    return;
  }

  req.userId = userId;
  next();
}
```

והשימוש:

```
app.use(parseUserId);
```

middleware בדוגמא מוסיף משתנה של userId לrequest. כל route שיוגדר לאחר מכן על app, ייקרא לאחר הפעלת middleware על request ולכן יכיל את הפרמטר userId ויוכל לגשת אליו.

בנוסף, ניתן להגדיר פונקציית middleware גנרית שמקבלת פרמטרים, באופן הבא:

```
export function logContext(context: string) {
  return function (req, res, next) {
    console.log(`New request on context: ${context}`);
    next();
  }
}
```

והשימוש:

```
app.get('/users', logContext('usersController'));
```

צורת שימוש נוספת בmiddleware, היא זימון של פונקציית middleware אחת או יותר לפני הקריאה לroute handler, באותה קריאה לapp.method. בשעת הקריאה לroute handler, middlewarees המוגדרים ירוצו לפי הסדר כשכל אחד רץ רק כשהmiddleware הקודם קורא לnext(). לדוגמא:

```
export function logRequest(req, res, next) {
  console.log(`[${req.method}] ${req.url}`);
  next();
}
```

```
app.get('/login', logRequest, (req: Request, res: Response) => {
  if (req.query.password == "abcd") {
    res.send("Success");
  } else {
    res.send("Denied")
  }
}));
```

בכל בקשה ל/login, הmiddleware של logRequest ירוץ ראשון על הבקשה, ורק כשיקרא לnext הroute handler עצמו ירוץ handler הבא של הבקשה.

אם הmiddleware לא קורא לnext ובעצם מסיים את הטיפול בבקשה, היא לא תועבר לניהול ע"י הmiddleware הבאים. לדוגמא:

```
export function isAuthorized(req, res, next) {
  if (req.userRole == "admin") {
    next();
  }

  res.status(403).end('User is not authorized to this resource');
}
```

```
usersRouter.put('/:userId', parseUserId, isAuthorized, (req, res) =>
{
  db.update(req.body.user);
  res.end();
});
```

הִisAuthorized middleware מסיים את ניהול הבקשה ולכן במידה והִuser לא מורשה לגשת לִresource, היא תסתיים בשגיאה והִroute handler לא ירוץ.

במידה והוגדרו יותר מִmiddleware או route handler אחד לאותו route, הם ייקראו לפי סדר הגדרתם.

ניתן להגדיר middleware גם ברמת הִrouter, כך שירוצו על כל הִroutes שמוגדרים בִrouter או על חלקם. אפשרות זו שימושית להגדרת מִmiddlewares על controller ספציפי כך שכל הִroutes תחתיו ינוהלו על ידו, לדוגמא הגדרת הרשאות לפי controller – ניהול הִauthorization לפי רמת ההרשאה של המבקש, ובדיקה אם הוא מורשה לִresource הספציפי שמנוהל ע"י הִcontroller.

Built-In Middlewares

בִexpress ישנם מִmiddlewares שהם built-in ושימושיים להגדרות שונות על הִexpress app.

לדוגמא, כדי לקרוא את הִbody של הִrequest, צריך לקבל את הִbody כִstream, ולהמיר אותו מִjson לִjavascript object. פעולה זו קיימת כִbuilt-in middleware בִexpress – express.json(). רק לאחר זימון הִmiddleware נוכל לקרוא לִreq.body בִroute handler ולקרוא את תוכן הבקשה:

```
app.use(express.json());

app.post('/products', (req, res) => {
  const product = req.body as Product;
  // handle the create operation
});
```

Built-in middlewares נוספים:

- express.static() – משמש להנגשת קבצים סטטיים מהשרת.
- express.urlencoded() – קורא request body שנשלח בפורמט של .urlencoded

Error Handling Middlewares

Express כולל ניהול שגיאות by default, כך שאם נזרקות שגיאות ב route handlers הן תחזורנה ל user עם status code המתאים וה server לא ייפול. ניתן להגדיר middleware שתופס את השגיאות הללו ומנהל אותן בהתאמה אישית.

הגדרת middleware היא באמצעות פונקציה שמקבלת ארבעה פרמטרים – כאשר הראשון הוא err שנתפס, והאחרים הם req, res ו next כרגיל:

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something broke in app!');  
});
```

ה error handling middleware תופס שגיאות שנזרקות ב handlers שהוגדרו לפניו, ולכן כדי לנהל שגיאות באופן גלובלי ב app יש לזמן אותו אחרון.

ניתן לכתוב error handling middleware ספציפי ל router מסוים, ולקרוא לו לאחר הגדרת routes ב router. במקרה כזה השגיאה תיפול ב middleware הפנימי ותנוהל בו, ולא תגיע לחיצוני גם אם קיים.

ניהול שגיאות נכון ב server הוא קריטי כדי לתעד את השגיאות, לנהל אותן ולהחזיר ל user כולל status code וה status message המתאימים. באמצעות שימוש ב error handling middleware ניתן למנוע שגיאות לא מנוהלות והחזרה של מידע לא מותאם ל user – מפורט מידי או לא אינפורמטיבי מספיק.

Third-Party Middlewares

ישנם middlewares שימושיים לפעולות נפוצות שניתן להתקין באמצעות חבילות npm ולהשתמש על ה express app.

דוגמא נפוצה ל middleware כזה היא cookie-parser לחילוץ cookies מתוך request.

Third-party middlewares נוספים: cors, cookie-session, connect-timeout.

מבנה פרויקט express

בכתיבת פרויקט nodejs עם typescript, הגדרת הserver עם express דורשת ניהול מושכל של מבנה התיקיות, הקבצים והשכבות. דרך נפוצה לחלוקה כזו היא לפי resources – יצירת תיקיה לכל resource, כאשר תחת התיקיה יושבים קבצים dal, service, api. חוץ מתיקיות הresources, ניתן להגדיר תיקיות שונות לtypes (לעיתים יושב תחת כל resource), consts (כנ"ל), middlewares וutilities.

מבנה התיקיות בפרויקט לדוגמא:

```

  ✓ middlewares
    TS log-request.ts
    TS parse-user.ts
  > node_modules
  ✓ products
    TS api.ts
    TS dal.ts
    TS service.ts
  ✓ types
    TS product.ts
  ✓ users
    TS api.ts
    TS dal.ts
    TS service.ts
  ✓ utils
    TS db-connect.ts
  TS app.ts
  TS main.ts
  {} package-lock.json
  {} package.json
```

קונבנציה נוספת שכדאי להיצמד אליה, היא שימוש בclasses בלבד – ללא קבצים שכוללים קוד להרצה או פונקציות בודדות. כך, קובץ app לדוגמא נראה כך:

```
import express from 'express';
import DbConnect from './utils/db-connect';
import ProductsDal from './products/dal';
import ProductsService from './products/service';
import ProductApi from './products/api';

const PORT = 5060;

export default class App {
  private app: express.Application;
  private productsApi: ProductApi;
  constructor() {}

  public async init() {
    this.app = express();
    this.app.use(express.json());

    const dbConn = new DbConnect();
    await dbConn.init();

    const productsDal = new ProductsDal(dbConn);
    await productsDal.init();
    const productsService = new ProductsService(productsDal);
    this.productsApi = new ProductApi(productsService);

    this.setRoutes();
  }

  private async setRoutes() {
    this.productsApi.setRoutes();
    this.app.use('/api/products', this.productsApi.router);

    this.app.listen(PORT, () => {
      console.log(`listening on port ${PORT}`);
    });
  }
}
```

כשקובץ Router:

```
import { Request, Response, Router } from "express";
import ProductsService from "../service";
import { Product } from "../types/product";
import { parseUser } from "../middlewares/parse-user";

export default class ProductApi {
  public router: Router;
  constructor(private productsService: ProductsService) {
    this.router = Router();
  }

  public setRoutes() {
    this.router.use(parseUser);
    this.router.get('/', async (req, res) => {
      const products = await
this.productsService.getAllProducts();
      res.send(products);
    });

    // all REST
  }
}
```

כדי להריץ את קובץ app, ניצור קובץ main שיוצר מופע של class ומריץ אותו. לדוגמא:

```
import App from "../app"

(async () => {
  const app = new App();
  await app.init()
})();
```

כעת, בהרצת קובץ main app יעלה עם כל routers המוגדרים עליו, וניתן יהיה לשלוח בקשות לserver ולקבל תשובות מהrouten המתאים.